Bridging Legacy Core Banking to Both U.S. Instant Payment Rails: FedNow and RTP

Daniel Stelzer Mori

Revised preprint · November 9, 2026

Abstract

The Federal Reserve's FedNow Service and The Clearing House's RTP network allow participating U.S. institutions to offer instant payments. Connecting to either rail requires bank-side integration that can coordinate interbank settlement with account posting, funds availability, and recovery from uncertain outcomes. This paper presents OpenFedNow, an open-source reference framework for examining those integration problems. Its architecture separates rail interfaces, core translation, payment orchestration, a maintenance-window experiment, and core adapters. A portable send reliability profile evaluates ordinary payments, lost confirmations, duplicate requests, and competing reservations using the same checks across two project-built synthetic implementations. Both implementations passed all four cases. A separate read-only Fiserv Premier Trial experiment verified authentication and retrieval of a published sample account. The results establish specific reference behaviors under controlled conditions. Production integration would require authenticated rail connectivity, authoritative bank-core controls, and institution-specific validation. The published driver and reports allow an outside engineer to attempt the same checks against a controlled implementation.

Keywords · Instant payments, FedNow, RTP, core banking, open-source middleware, payment reliability, software testing

Revision note · This version updates the working paper first posted on Zenodo and SSRN in July 2026. It distinguishes reference designs and fixture tests from observations made in a provider environment. Repository claims refer to the November 9 snapshot unless a different test revision is identified. [1, 2]

1. The bank-side integration problem

FedNow and RTP are separate U.S. instant payment services with their own access arrangements, message profiles, and operating rules. At a financial institution, enabling a payment also involves its account systems, customer channels, risk controls, and operations. The Federal Reserve Bank of Kansas City describes how cores and operating processes designed for earlier payment models can make automated, continuous processing difficult and expensive. It identifies several integration approaches, including modernizing core components and using a payment hub to connect existing systems. [3-5]

The 2026 Conference of State Bank Supervisors survey illustrates a difference between receive and send availability. Among respondents to the respective questions, 38.2% said they offered FedNow receipt and planned to continue, while 14.4% said the same for sending. Another 34.3% of respondents to the send question reported plans to introduce sending within twelve months. These figures describe the participating community banks' reported offerings and plans. A Federal Reserve Financial Services

interview separately documents one provider dependency. AdventHealth Credit Union reported onboarding as a receiver in five days and said that it was waiting for its core provider to enable send. [6, 7]

Receive and send have different dependencies. Receiving requires timely validation, customer credit, immediate funds availability under the applicable rules, and reconciliation. Sending adds a customer initiation path, authorization and fraud controls, and a reliable way to prevent the same funds from being spent through competing channels. Both depend on the institution's settlement and operating arrangements. The engineering question examined here is how to separate reusable payment behavior from the particular rail transport, core API, or account authority that supplies those guarantees. [3, 4]

2. Design lessons from Pix

In a prior role at IBM, the author led architectural work for Santander Brazil's internet-banking Pix integration and oversaw front-end and back-end implementation. That work included send and receive flows, Pix key management, and integration with Banco Central do Brasil's Instant Payments System, or SPI. When a payment remained uncertain, the channel showed a pending state and checked its status later rather than offering a new attempt that could create a second payment. This account describes the author's work on the Santander integration. The central bank's flow manual provides separate documentation of the Pix scheme. [8]

The official immediate interbank flow distinguishes the payer institution's reservation on the customer account, settlement between participants, and final posting to the customer accounts. The distinction is useful when designing a U.S. integration. For example, an application may receive evidence of interbank settlement while it still needs to account for a local posting result. Payment identity, account effects and status observations therefore belong in the design from the start. Pix, FedNow and RTP each require their applicable network specifications and operational controls. [3, 5, 8]

3. OpenFedNow architecture and implementation scope

OpenFedNow explores a shared middleware design for legacy-core integration. The original architecture groups responsibilities into five layers. Rail gateways handle the network-facing message boundary. A translation layer maps between shared payment concepts and core-specific representations. A Shadow Ledger and Availability Bridge model a core interruption. Payment orchestration tracks the lifecycle and coordinates effects. Vendor adapters isolate product-specific operations. The source is available under Apache 2.0. [2, 9]

The evaluated outbound workflow is a distinct SQL-based reference path. It durably claims a payment by scoped identity, checks for conflicts if the same identity is reused with changed details, and reserves funds in a synthetic PostgreSQL account before dispatch. After it records that a send may have occurred, a missing response leaves the payment pending with the reservation intact. The workflow queries the simulator and applies one local posting upon confirmed synthetic settlement, or releases the hold upon confirmed rejection. The service declines to send when the fixture lacks exclusive control over the account's debit channels. The non-production `/fednow/send` bridge can route to this same SQL service. [9, 10]

The Redis Shadow Ledger and Availability Bridge exercise a separate inbound maintenance scenario. The fixture queues work during simulated core unavailability and reconciles local state when its core-shaped adapter is available again. The cached balance and the SQL audit record have separate

commit boundaries, and a cache can miss debits made through another channel. Production receiving during core maintenance would require timely responses and immediate customer funds availability. When planned downtime prevents either, the current FedNow procedures require the participant to sign off for receipt. A provisional entry in the fixture is an observation of the local design, while immediate spendability is a separate bank-side outcome to validate. [3, 9, 11]

The existing FIS-, Fiserv-, and Jack Henry-shaped adapters exercise selected request and response mappings using local fixtures. The separate Fiserv Banking Hub Premier work verified authentication and a read of a published synthetic deposit account in the shared Trial environment. That observation establishes access to the read operation. Available-balance semantics, reservations across debit channels, and financial posting remain validation work. The RTP reference gateway exercises selected message parsing and serialization. At the reviewed revision, its outbound route returns an unavailable response without sending to a rail. [9, 12]

4. Portable send evaluation

The version 1.0.0 send profile selects four scenarios from the repository's broader evaluation. A driver maps the required operations to a test target, while the shared case logic evaluates expected effects. The rail simulator runs as a separate process and records submissions and synthetic outcomes. This separation lets the evaluator compare the target's reported payment and account state with the simulator's record. The published runner includes an example driver and produces a structured report that identifies the profile, runner, shared test logic, target revision, and environment. Results are classified as passed, failed, or blocked when a required capability or observation is unavailable. [13]

Scenario	Financial or operational behavior evaluated	Java / PostgreSQL	Python / SQLite
S01 · Settlement and rejection	Each payment has one submission. Settlement posts the hold once. Rejection releases it.	Pass	Pass
S02 · Lost confirmation	A hold survives an uncertain outcome. A duplicate request retrieves the same operation. Inquiry resolves the original payment with one submission and one posting.	Pass	Pass
S06 · Concurrent duplicate request	Identical concurrent requests resolve to one operation and one financial effect.	Pass	Pass
S16 · Competing payments	Two distinct payments compete for an insufficient combined balance. Only one proceeds in the exclusive-account fixture.	Pass	Pass

Table 1. Project-run results for the portable send profile. Both targets and the rail simulator used synthetic accounts and test controls. The manifest identifies the evaluated versions, commands and checksums. [13]

In S02, each target began with a synthetic booked balance of $1,000. It placed a $100 hold and retained the $1,000 booked balance while the payment outcome was uncertain. A repeated request resolved to the same operation. After inquiry, the reports showed a $900 booked balance, a zero hold, and one simulator submission. The archived runner tests also exercise deliberate duplicate submission, mismatched financial effects, unavailable observations, and report redaction. These results describe the

tested scenarios and schedules. Evaluation in a third-party environment, authenticated live status, and enforcement across a real bank's debit channels remain open. [13]

The lost-confirmation case reflects a problem recognized in the FedNow procedures. When message status is uncertain after the applicable timeout, the procedures describe checking status before submitting another message. The local profile uses a JSON simulator with its own timing. The test concerns preservation of payment identity and funds while the outcome remains uncertain. An authorized participant would need to implement inquiry and status authentication under the applicable rail specifications. [3, 10, 13]

5. Use by institutions and providers

The proposed workflow is to run a published scenario, map the target-specific operations through a driver, and compare the observed account and rail effects using a common report. A provider serving multiple institutions could apply the same checks across its controlled implementations. A bank could use a blocked result to identify an operation it needs its core provider to expose, such as a recoverable reservation or a way to observe a posting independently. These are prospective uses of the package. [3, 13]

The UK Open Banking Functional Conformance Tool illustrates a distribution model for shared tests. Open Banking Limited supplies a common tool and publishes results from its formal certification program. OpenFedNow applies the reusable testing pattern to selected bank-side reliability scenarios. The FedNow Service's customer testing and certification process remains separate. [3, 14]

A next evaluation should have an outside engineer run an unchanged send case against a target they control. The report should identify the driver, the target revision, what could be observed, and what required a change. A receive-side profile can start with duplicate credit and reconciliation and then examine a maintenance interruption with an independently observable spendable balance. Institution-specific work would also have to establish authorized rail access, authoritative balance and hold semantics, security and fraud controls, returns, reconciliation, and operational ownership. [3, 9-13]

6. Conclusion

OpenFedNow provides a reference architecture and a reproducible way to evaluate a bounded set of bank-side send controls. The current evidence consists of matching results across two project-built targets and a separate read-only provider Trial observation. The path to broader use is to establish that an outside engineer can apply an unchanged scenario to a different implementation, then document where the framework's reusable behavior fits the institution's operating and integration requirements. [9, 12, 13]

Funding · This work was conducted independently and received no external funding.

References

1. Mori, D. *Bridging Legacy Core Banking to Both U.S. Instant Payment Rails: FedNow and RTP*. Original preprint, dated May 13, 2026. SSRN record and DOI · Zenodo record and DOI.
2. Mori, D. *OpenFedNow*. Source repository and license. November 9, 2026 source snapshot.
3. Federal Reserve Financial Services. *FedNow Service Operating Procedures*, version 3.7, October 2026. Customer Testing Environment, Participant Availability, and message duplication and inquiry provisions. Operating procedures.
4. Federal Reserve Bank of Kansas City. *The Role of Core Banking Services Providers in Facilitating Instant Payments*, May 8, 2024. Research briefing.
5. The Clearing House. *RTP Network*. Service overview.
6. Conference of State Bank Supervisors. *2026 Annual Survey of Community Banks*, Figure 9. Survey report.
7. Federal Reserve Financial Services. *High-speed onboarding: Credit union goes live on FedNow Service in record time*, June 3, 2026. AdventHealth interview.
8. Banco Central do Brasil. *Manual de Fluxos do Processo de Efetivação do Pix*, §§2-3. Pix flow manual.
9. OpenFedNow. *Architecture Overview* and *Capability and Evidence Matrix*. Architecture · Capability matrix.
10. OpenFedNow. *Synthetic Reliability Reference Architecture*. Design and state transitions.
11. OpenFedNow. *Redis Shadow Ledger: Sandbox Behavior and Limits*. Scope and reconciliation behavior.
12. OpenFedNow. *Fiserv Banking Hub Integration Discovery*. Trial evidence and limits · PR 107.
13. OpenFedNow. *Portable Send Reliability Profile v1*, November 9, 2026. Profile guide · Evidence manifest and reports · PR 110.
14. Open Banking Limited. *Functional Conformance* and *Functional Conformance Tool*. Certification results · Tool repository.